LDA and STM in R Example

Contents

How the whole thing "flows"

1. **One preprocessing pipeline** (chunk 3) feeds **both** LDA and STM so we're comparing apples to apples.
2. LDA section: **fit → top words → top docs → quick labels** (chunks 5–7).
3. STM section: **fit → labels → top docs → contrasts** (chunks 8–11), then a **no-content** STM to show the four scoring lists (chunk 12).

1) `setup`

- **What it does:** Quiet R Markdown messages and set a fixed random seed.
- **Why:** Keeps the PDF tidy and makes results reproducible.
- **Outputs:** Nothing user-facing; global knitr settings + `set.seed(42)`.

```
knitr::opts_chunk$set(message = FALSE, warning = FALSE)
set.seed(42)
```

2) *(packages)*

- **What it does:** Ensures required packages are installed, then loads them quietly.

- **Why:** Guarantees the rest of the file can run without missing libraries.
- **Outputs:** Loaded packages in memory (quanteda, topicmodels, stm, dplyr, kableExtra, etc.).

```r
# Ensure required packages are installed, then load them
options(repos = c(CRAN = "https://cloud.r-project.org"))

req_pkgs <- c(
  "quanteda", "quanteda.textstats", "quanteda.textmodels",
  "topicmodels", "SnowballC", "kableExtra",
  "dplyr", "tidyr", "stringr", "purrr", "Matrix",
  "readr", "tibble", "knitr", "stringi", "stm"
)

to_install <- setdiff(req_pkgs, rownames(installed.packages()))
if (length(to_install)) {
  install.packages(to_install, dependencies = TRUE)
}

# Load quietly
invisible(lapply(req_pkgs, function(p) {
  suppressPackageStartupMessages(library(p, character.only = TRUE))
}))
```

3) `preprocess_unified`

- **Inputs:** A raw text file `article.txt` (one article) in the working folder.
- **What it does (big picture):**
  - Sets **global knobs**: number of topics `K`, how many top words/docs to show, a frequency cutoff for pruning rare words, and whether to stem words.
  - Defines **domain stopwords** (e.g., "like", "people") to remove unhelpful filler.
  - **Reads and splits** the article into **paragraphs** (the "documents") and builds simple **metadata** per paragraph: its index (`doc_id`), position in the article, length, and a coarse **section** label (early/middle/late).
  - Runs STM's **`textProcessor`** and **`prepDocuments`** to tokenize, clean, and **trim** the vocabulary by frequency.
  - Builds two matrix "bridges" from the same cleaned data:
    - a **quanteda dfm** for quanteda workflows. `quanteda`is used only as a container/adapter: we wrap the STM-prepped counts into a `dfm` (`dfm_from_prep`) and then `convert()` that to a `topicmodels::DocumentTermMatrix` (`dtm_topicmodels`).
    - a **topicmodels DTM** for `topicmodels::LDA`. `topicmodels` is used to **fit LDA** on `dtm_topicmodels` and to extract results (`posterior(lda_fit)$topics` for θ, `terms(lda_fit, n)`/`posterior(...)$terms` for top words).
  - Saves a vector that maps **kept rows back to the original paragraph IDs**.
- **Outputs (named objects):**
  - `prep$documents`, `prep$vocab`, `prep$meta` (for STM).
  - `dfm_from_prep` (quanteda dfm) and `dtm_topicmodels` (for LDA).
  - `paragraphs` (the cleaned text vector) and `orig_ids` (row→original paragraph map).
- **Used next by:** LDA fit (chunk 5) and all STM chunks (8–12).

The part about 2 matrix "bridges" is a bit confusing. By "matrix bridge," I mean a tiny adapter layer that takes one canonical bag-of-words count table and presents it in whatever container a library expects—so every library sees the same numbers and vocabulary even though they use different R classes.

In the pipeline, STM's `prepDocuments()` doesn't give a rectangular table; it gives per-document term indices and counts. It first reassembles those into a single sparse D×V (documents x vocab) document-term matrix (rows = kept paragraphs, columns = the shared prep$vocab). That sparse matrix is the canonical representation of the data. From there, we create bridges:

- wrap the same counts as a quanteda dfm (so quanteda tools work), and
- wrap the same counts as a topicmodels::DocumentTermMatrix (so topicmodels::LDA() works).

All three objects: the sparse matrix, the dfm, and the DTM hold identical counts with identical column order; they're just different "jackets" around the same numbers. That's the whole point of the bridge: no re-tokenizing, no re-pruning, no drift. LDA and STM compare apples to apples.

Why save `orig_ids`?

`prepDocuments(...)` **drops** any paragraphs that become empty after cleaning/trimming and reorders metadata to match the kept documents. That means "row 17" in `prep$documents` is not necessarily "paragraph 17" from the original `paragraphs` vector.

We set `orig_ids <- prep$meta$doc_id`. Now:

- `orig_ids[d]` tells us which **original paragraph index** ended up as **row d** in the kept matrix.
- Later, when we pick top-documents by topic using `theta_lda` or `stm_fit$theta` (both are D×K, aligned with `prep$documents`), we map back to human-readable text via `paragraphs[ orig_ids[doc_id] ]`. Without `orig_ids`, we'd print the wrong snippet after trimming.

Coordinate list (also called triplet) representation

A tiny toy picture

Suppose `vocab = c("cat","dog","tree")` so `V=3`. Two kept paragraphs (`D=2`):

- doc 1 has "cat cat dog" -> term indices `j=[1,2]`, counts `x=[2,1]` (cat twice, dog once);
- doc 2 has "tree" -> `j=[3]`, `x=[1]`.

Flattening gives `i_idx = [1,1,2]`, `j_idx = [1,2,3]`, `x_val = [2,1,1]`.

To see how this works:

- `i_idx = [1, 1, 2]` — row (document) indices for each **non-zero** cell. Read it as: "there's a non-zero in **doc 1**, another non-zero in **doc 1**, and a non-zero in **doc 2**." (R is 1-based.) The reason doc 1 appears twice is that it has **two distinct terms** with non-zero counts.
- `j_idx = [1, 2, 3]` — column (term) indices into the vocabulary. Read alongside `i_idx`: the first non-zero is for **term 1 = cat** in doc 1; the second is **term 2 = dog** in doc 1; the third is **term 3 = tree** in doc 2.
- `x_val = [2, 1, 1]` — the counts for those (doc, term) cells: (doc1, cat) has count **2**, (doc1, dog) has **1**, (doc2, tree) has **1**.

Put them together (by position `k`) to see the triples:

- `k=1`: `(i=1, j=1, x=2)` → (doc1, cat) = 2
- `k=2`: `(i=1, j=2, x=1)` → (doc1, dog) = 1
- `k=3`: `(i=2, j=3, x=1)` → (doc2, tree) = 1

To build the sparse matrix, think of the three vectors as **triples** that tell us where to *add* counts in a blank D×V table (rows = documents, columns = vocab). Here D=2 (doc1, doc2) and V=3 (cat, dog, tree).

1. Take the first triple `(i=1, j=1, x=2)`. Put **2** in **row 1, col 1** → (doc1, cat) = 2.

2. Second triple `(i=1, j=2, x=1)`. Put **1** in **row 1, col 2** → (doc1, dog) = 1.
3. Third triple `(i=2, j=3, x=1)`. Put **1** in **row 2, col 3** → (doc2, tree) = 1.

All other cells stay **0** (because they never appear in the triplets). If a document–term pair appeared more than once in `(i_idx, j_idx)`, you'd **add** the `x_vals` together at that cell.

`sparseMatrix(i,j,x,dims=c(2,3))` yields

```
      cat dog tree
doc1    2   1    0
doc2    0   0    1
```

as.dfm(...) wraps that as a dfm (document feature matrix); convert(..., to="topicmodels") wraps it as a DTM (document term matrix). DFM and DTM are basically synonyms and the difference is just how they're wrapped as implementations for the libraries. If the original paragraphs were length 3 and prep dropped paragraph 2, then orig_ids = c(1,3) so row 1 maps back to paragraphs[1], row 2 to paragraphs[3].

```
# Global knobs with safe defaults
if (!exists("K"))            K <- 5                # requested number of topics
if (!exists("N_TOP_WORDS"))  N_TOP_WORDS <- 10     # words to show per topic
if (!exists("N_TOP_DOCS"))   N_TOP_DOCS <- 2       # top docs/snippets per topic
if (!exists("LOWER_THRESH")) LOWER_THRESH <- 2     # drop tokens with total count < this
if (!exists("DO_STEM"))      DO_STEM <- FALSE      # don't apply stemming in tokenization by default

# Domain stopwords beyond the built-in English list that we found a lot in the article
if (!exists("domain_stop")) {
  domain_stop <- c(
    "like","people","human","humans","really","just",
    "make","made","many","much","things","thing",
    "today","officials","report","reported","says",
    "can","one"
  )
}

# Load or reuse paragraphs (one paragraph = one "document")
if (!exists("paragraphs")) {
  txt <- readr::read_file("article.txt")
  paragraphs <- stringr::str_split(txt, "\\n\\s*\\n")[[1]] |> # split on blank lines
    stringr::str_replace_all("\\s+", " ") |>                 # normalize whitespace (ie make it all a
    stringr::str_trim()
  paragraphs <- paragraphs[paragraphs != ""]                 # drop empty paragraphs
}

# Build document-level metadata used by STM:
# - doc_id: stable ID of the original paragraph
# - position: order in article
# - length: token count per paragraph
# - section: coarse terciles (early/middle/late) for content covariate
meta <- tibble::tibble(
  doc_id   = seq_along(paragraphs),
  position = seq_along(paragraphs),
  length   = stringr::str_count(paragraphs, "\\S+")
) |>
  dplyr::mutate(
    section = cut(
      position,
```

```
      breaks = stats::quantile(position, probs = c(0, 1/3, 2/3, 1), na.rm = TRUE),
      include.lowest = TRUE,
      labels = c("early", "middle", "late")
    )
  )

# Tokenize/clean with STM helpers; apply stemming + stopword removal + pruning later
proc <- stm::textProcessor(
  documents          = paragraphs,
  metadata           = meta,
  lowercase          = TRUE,
  removestopwords    = TRUE,
  removenumbers      = TRUE,
  removepunctuation  = TRUE,
  stem               = DO_STEM,
  customstopwords    = domain_stop
)
```

```
## Building corpus...
## Converting to Lower Case...
## Removing punctuation...
## Removing stopwords...
## Remove Custom Stopwords...
## Removing numbers...
## Creating Output...
```

```
# Construct STM's internal sparse format and trim rare terms (lower.thresh)
# This may drop empty documents; it also reorders/filters to "kept" docs
prep <- stm::prepDocuments(proc$documents, proc$vocab, proc$meta,
                           lower.thresh = LOWER_THRESH)
```

```
## Removing 2680 of 3562 terms (3233 of 9524 tokens) due to frequency
## Removing 7 Documents with No Words
## Your corpus now has 324 documents, 882 terms and 6291 tokens.
```

```
# Map rows of kept docs back to original paragraph indices (for snippet lookup later)
orig_ids <- prep$meta$doc_id

# Bridges for LDA / quanteda
D <- length(prep$documents)
V <- length(prep$vocab)

# Each prep$documents[[d]] is a 2×m matrix: row1 = term indices, row2 = counts
nnz_per_doc <- vapply(prep$documents, ncol, integer(1))                    # nonzeros per doc
i_idx <- rep(seq_len(D), times = nnz_per_doc)                              # row indices (doc IDs)
j_idx <- unlist(lapply(prep$documents, function(m) m[1, ]), use.names = FALSE) # col indices (term IDs)
x_val <- unlist(lapply(prep$documents, function(m) m[2, ]), use.names = FALSE) # counts

# Canonical sparse document-term matrix with vocab as column names
# "Canonical" = the single, source-of-truth D×V count table: rows = kept documents, cols = vocabulary t
# It's "sparse" because only non-zero counts are stored; all other word-doc pairs are implicit zeros.
# Column names are exactly prep$vocab, fixing the column order so every downstream step sees identical
# This matrix is the basis; the quanteda dfm and topicmodels DTM are just wrappers around the same coun
# Building it from STM's per-doc indices ensures LDA and STM use identical tokenization and pruning.
dtm_sparse <- Matrix::sparseMatrix(
```

```
  i = i_idx,
  j = j_idx,
  x = x_val,
  dims = c(D, V),
  dimnames = list(NULL, prep$vocab)
)
# Two thin "bridges" (adapters) so other packages see the exact same counts/vocab
dfm_from_prep   <- quanteda::as.dfm(dtm_sparse)                          # for quanteda workflows
dtm_topicmodels <- quanteda::convert(dfm_from_prep, to = "topicmodels") # for topicmodels::LDA

# Objects produced:
# - prep$documents, prep$vocab, prep$meta  -> use for stm()
# - dfm_from_prep (quanteda dfm)           -> use for quanteda textmodels
# - dtm_topicmodels (topicmodels DTM)      -> use for topicmodels::LDA()
# - orig_ids                               -> use to index back to original paragraphs
```

4) helpers

This section provides two small utilities used throughout the tutorial. `top_docs_for_topic()` takes a **document–topic matrix** θ and, for a chosen topic k, returns the **top N paragraphs** with short, readable snippets. Here θ is a D×K matrix of document–topic weights where $\theta_{d,k} \approx P(\text{topic k} \mid \text{document d})$; for LDA this comes from `posterior(lda_fit)$topics`, and for STM it's `stm_fit$theta`. `fmt_table()` centralizes table styling (captions, alignment, column widths) so every table looks consistent without repeating kable/kableExtra boilerplate.

- **What it does:** Defines two utilities:
  - `top_docs_for_topic()`: given a topic's document-probabilities, it finds the **top N paragraphs** for that topic and makes **short snippets**.
  - `fmt_table()`: one place to style kable tables consistently (caption, widths, alignment).
- **Outputs:** Two helper functions in memory.
- **Used next by:** LDA/STM "top docs" tables and most tables that follow.

```
# Helper: return the top-N documents for topic k, with human-readable snippets.
# Inputs:
#   - theta_mat: D×K document-topic matrix (theta), e.g. posterior(lda_fit)$topics or stm_fit$theta
#   - k:         1-based topic index to rank by
#   - topn:      how many documents to return
#   - map_ids:   optional integer vector of original paragraph IDs (e.g., orig_ids);
#                if provided, results report those original IDs instead of row indices
#   - paras:     character vector of paragraph texts indexed by original IDs (for snippets)
# Output: tibble with original Doc ID, the topic probability theta_{d,k}, and a 260-char snippet
top_docs_for_topic <- function(theta_mat, k, topn = 2, map_ids = NULL, paras = paragraphs) {
  # Rank documents by theta_{d,k} descending and keep the top N (guard against short corpora)
  ord <- order(theta_mat[, k], decreasing = TRUE)
  ord <- ord[seq_len(min(topn, length(ord)))]

  # Choose which IDs to display: original paragraph IDs if provided, else row numbers
  ids <- if (is.null(map_ids)) ord else map_ids[ord]

  # Build short, readable snippets from the original paragraphs (indexed by original IDs)
  snips <- ifelse(nchar(paras[ids]) > 260,
                  paste0(substr(paras[ids], 1, 260), "..."),
                  paras[ids])
```

```
  tibble(
    `Doc ID` = ids,
    prob     = round(theta_mat[ord, k], 3),  # theta_{d,k} for transparency
    snippet  = snips
  )
}

# Column-width preset used by 4-column tables (e.g., topic, doc id, prob, snippet)
WIDTHS_4 <- c("1.5cm","1.5cm","1.5cm","10cm")

# Single source of truth for table formatting so all tables share a consistent style.
# - caption:     table caption
# - align:       optional vector of column alignments; default centers all but last (left)
# - widths:      optional vector of explicit column widths (same length as ncol(df))
# - latex_opts:  kableExtra options; "hold_position" helps LaTeX place longtables sensibly
fmt_table <- function(df, caption, align = NULL, widths = NULL,
                      latex_opts = c("striped","hold_position")) {
  # Default alignment: center all columns except the last, which is left (better for text)
  if (is.null(align)) {
    n <- ncol(df)
    align <- if (n >= 2) c(rep("c", n - 1), "l") else rep("c", n)
  }

  # Build the kable and apply styling once here (DRY)
  out <- kbl(df, caption = caption, booktabs = TRUE, escape = TRUE,
             longtable = TRUE, align = align) |>
         kable_styling(latex_options = latex_opts)

  # Optional fixed widths (defensive check so widths match columns)
  if (!is.null(widths)) {
    stopifnot(length(widths) == ncol(df))
    for (j in seq_along(widths)) out <- column_spec(out, j, width = widths[[j]])
  }
  out
}
```

Purpose

We're going to dive into something very similar to problem 1 from the homework using **R** with an article (each paragraph = one document), run a single preprocessing pipeline, then fit **both LDA** (topicmodels) **and STM** (with prevalence and a content covariate), using the **same number of topics K** as in Python, and compare their outputs.

- For **LDA**: Top **10 words** per topic; **Top 2 paragraphs** most associated with each topic; a short **2–3 word label** per topic (auto-derived, editable).
- For **STM**: Top **10 words** per topic (SAGE/FREX as appropriate); **Top 2 paragraphs** per topic; content-level **perspectives** across early/middle/late; plus an appendix with **Prob / FREX / Lift / Score** lists.

Topics are ordered by **prevalence** so LDA and STM tables share a consistent topic numbering for comparison.

LDA (topicmodels)

Set **K** as we previously used for this problem in Python. Fit a baseline LDA on the trimmed DTM and order topics by prevalence so all downstream tables use a consistent topic order.

5) `fit_lda`

This step fits a baseline LDA to the trimmed `dtm_topicmodels` using the same **K** as in the Python run. The Gibbs sampler is fixed-seed for reproducibility. After fitting, the **document–topic weight matrix** from `posterior(lda_fit)$topics` is used to compute a simple **prevalence order** (mean topic weight across documents) so every downstream table refers to "Topic 1, 2, ..." consistently. Top words per topic are then extracted and re-ordered to match that prevalence.

- **Inputs:** `dtm_topicmodels` from preprocessing and `K`.
- **What it does:** Fits an **LDA** model (Gibbs sampling), then:
  - Gets **posterior** topic probabilities per document (`theta_lda`).
  - Orders topics by **prevalence** (most common first).
  - Extracts **top 10 words** per topic and reorders columns to match that prevalence order.
  - Prints a **"Top 10 words per topic"** table.
- **Outputs:** `lda_fit`, `theta_lda`, `k_order`, and a top-words table.
- **Used next by:** LDA "top documents" and auto-labels.

```r
K <- 5  # keep K consistent with a previous the Python run

# Gibbs sampler controls:
# - seed: reproducibility
# - alpha: document-topic prior (smaller => sparser mixtures)
# - burnin/iter/thin: warmup, total iterations, and thinning rate
ctrl <- list(seed = 42, alpha = 0.5, estimate.beta = TRUE,
             verbose = 0, burnin = 2000, iter = 1000, thin = 100)

# Fit baseline LDA on the trimmed DTM (same counts/vocab that STM saw)
lda_fit <- LDA(dtm_topicmodels, k = K, method = "Gibbs", control = ctrl)

# Cache posterior once for the whole LDA section:
# - post_lda$topics: D×K document-topic weights (approx P(topic k | document d))
# - post_lda$terms : K×V topic-term probabilities (not used directly below)
post_lda  <- posterior(lda_fit)
theta_lda <- post_lda$topics

# Order topics by corpus prevalence so all later tables share the same numbering.
# (Prevalence = column mean of the document-topic weights.)
k_order <- order(colMeans(theta_lda), decreasing = TRUE)

# Extract top 10 words per topic and reorder columns to match the prevalence order
terms_mat_lda     <- terms(lda_fit, 10)                    # words × topics (topics are columns)
terms_mat_lda_ord <- terms_mat_lda[, k_order, drop = FALSE]

# Build a compact table: topic index (by prevalence) + space-joined top words
top_terms_lda <- tibble(
  topic     = seq_along(k_order),                          # 1..K in prevalence order
  top_words = apply(terms_mat_lda_ord, 2, function(col) paste(col, collapse = " "))
)
```

```r
# Render the summary; downstream chunks (top docs, labels) use the same topic order
kable(top_terms_lda, caption = "Top 10 words per topic (prevalence order descending)")
```

Table 1: Top 10 words per topic (prevalence order descending)

topic	top_words
1	computer brain intelligence even far able get power machine will
2	're life species 'll asi good beam know see don't
3	asi will agi first intelligence 'll likely question bostrom smarter
4	future time kurzweil years world 're happen believes progress might
5	turry goal way ani also get new systems work another

Reading down the top-word lists: Topic 1 centers on computing/brain capability (computer, brain, intelligence, machine, power). Topic 2 mixes conversational tokens ('re/'ll) with life/species/ASI. Topic 3 is about AGI/ASI and expert debate (ASI/AGI, likely, question, Bostrom). Topic 4 is the Kurzweil-style time/progress theme (future, time, years, progress). Topic 5 anchors the Turry vignette and goal/misalignment (turry, goal, systems, ani).

Most-associated paragraphs (top 2 per topic)

We use posterior **P(topic | doc)** to rank paragraphs, then show short snippets. Pick representative paragraphs by ranking docs with P(topic | doc); show short snippets (map to original paragraph IDs).

6) `lda_top_docs`

- **Inputs:** `theta_lda`, `k_order`, `orig_ids`, `paragraphs`.
- **What it does:** For each (prevalence-ordered) topic, finds the **top 2 paragraphs** and shows **short snippets**.
- **Outputs:** Nicely formatted table of **top paragraphs per LDA topic**.
- **Used next by:** Human interpretation; no downstream dependencies.

```r
# Reuse cached objects from fit-lda: theta_lda, k_order, and orig_ids/paragraphs
top_docs_lda <- purrr::map_dfr(seq_along(k_order), function(i) {
  k_orig <- k_order[i]
  top_docs_for_topic(theta_lda, k_orig, topn = 2, map_ids = orig_ids, paras = paragraphs) |>
    dplyr::mutate(topic = i) |>
    dplyr::select(topic, `Doc ID`, prob, snippet)
})

fmt_table(
  top_docs_lda,
  caption = "Top 2 paragraphs per topic (prevalence order)",
  align   = c("c","c","c","l"),
  widths  = WIDTHS_4
)
```

Table 2: Top 2 paragraphs per topic (prevalence order)

topic	Doc ID	prob	snippet

1	194	0.826	The fact is, aging isn't stuck to time. Time will continue moving, but aging doesn't have to. If you think about it, it makes sense. All aging is is the physical materials of the body wearing down. A car wears down over time too—but is its aging inevitable? If...
1	47	0.815	On the other hand, multiplying big numbers or playing chess are new activities for biological creatures and we haven't had any time to evolve a proficiency at them, so a computer doesn't need to work too hard to beat us. Think about it—which would you rather d...
2	218	0.838	You can see that the label "existential risk" is reserved for something that spans the species, spans generations (i.e. it's permanent) and it's devastating or death-inducing in its consequences. It technically includes a situation in which all humans are perm...
2	152	0.829	We'll talk more about the Main Camp in a minute, but first—what's your deal? Actually I know what your deal is, because it was my deal too before I started researching this topic. Some reasons most people aren't really thinking about this topic:
3	141	0.911	Müller and Bostrom also asked the experts how likely they think it is that we'll reach ASI A) within two years of reaching AGI (i.e. an almost-immediate intelligence explosion), and B) within 30 years. The results:4
3	135	0.852	Median optimistic year (10% likelihood): 2022 Median realistic year (50% likelihood): 2040 Median pessimistic year (90% likelihood): 2075
4	10	0.926	This pattern—human progress moving quicker and quicker as time goes on—is what futurist Ray Kurzweil calls human history's Law of Accelerating Returns. This happens because more advanced societies have the ability to progress at a faster rate than less advance...
4	9	0.908	In order for someone to be transported into the future and die from the level of shock they'd experience, they have to go enough years ahead that a "die level of progress," or a Die Progress Unit (DPU) has been achieved. So a DPU took over 100,000 years in hun...
5	239	0.863	To build Turry's writing skills, she is programmed to write the first part of the note in print and then sign "Robotica" in cursive so she can get practice with both skills. Turry has been uploaded with thousands of handwriting samples and the Robotica enginee...
5	236	0.862	The team at Robotica thinks Turry could be their biggest product yet. The plan is to perfect Turry's writing mechanics by getting her to practice the same test note over and over again:

The top paragraphs make those themes concrete. Topic 1 pairs a passage on aging/time with a reflection on computers beating us at tasks like chess. Topic 2 juxtaposes the definition of existential risk with a "why people aren't thinking about this" setup. Topic 3 surfaces the survey timing results for AGI/ASI. Topic 4 highlights Kurzweil's accelerating-returns and the "Die Progress Unit" thought experiment. Topic 5 anchors on the Turry handwriting/practice scene.

Auto-labels (2–3 words)

This step creates quick, human-readable labels for each LDA topic so tables and figures can reference "Topic 1: ..." with a short name. The idea is intentionally simple: take the first 2–3 tokens from each topic's top-words list and join them with slashes. We can treat these as placeholders; they're for navigation, not ground truth.

7) `labels`

- **Inputs:** The LDA **top-words** table.
- **What it does:** Creates quick **topic labels** by taking the first 2–3 top words (e.g., "economy/market/price").
- **Outputs:** A compact **auto-labels** table.
- **Used next by:** Human interpretation; no downstream dependencies.

```
# Build short, human-readable labels from the LDA top-words table.
# Strategy: normalize spaces -> split into tokens -> take first up to 3 -> join with "/".
auto_labels <- top_terms_lda |>
  dplyr::mutate(
    # Collapse any multiple spaces to a single space (defensive cleanup)
    label_src = stringr::str_replace_all(top_words, "\\s+", " "),
    # Split into tokens and keep the first up to 3; join with "/" for compactness
    label = purrr::map_chr(strsplit(label_src, " "),
                           ~ paste(head(.x, 3), collapse = "/"))
  ) |>
  dplyr::transmute(topic, label)

# Render a compact index of topic labels.
kable(auto_labels, caption = "Auto labels (first 2-3 top words)")
```

Table 3: Auto labels (first 2–3 top words)

topic	label
1	computer/brain/intelligence
2	're/life/species
3	asi/will/agi
4	future/time/kurzweil
5	turry/goal/way

The auto-generated labels match the themes we see elsewhere: Topic 1 ("computer/brain/intelligence") is about machine capability vs. human cognition; Topic 2 ("'re/life/species") blends conversational tokens ('re) with biology terms; Topic 3 ("asi/will/agi") centers on AGI/ASI and "will"/timing language; Topic 4 ("future/time/kurzweil") is the acceleration/timeline motif; Topic 5 ("turry/goal/way") anchors the Turry vignette and goal-specification/misalignment thread.

Notes

- Stemming vs. no stemming will change the top words (e.g., *intellig* vs *intelligence*). I've elected not to use stemming for readability and it didn't seem to improve results.
- If the vocabulary is **too small** or **too large**, adjust `lower.thresh` via `stm::prepDocuments(...)`
- That is, lower `LOWER_THRESH` above; e.g., `LOWER_THRESH <- 6` or `10`).

- Results will **not** exactly match Python's LDA due to different implementations/seeds, but they should be comparable.

8) 'Fit STM

Fit an STM on the same preprocessed corpus so results are comparable to LDA. The **prevalence** formula models where/when a paragraph appears and how long it is; the **content** formula allows each topic's word usage to shift across sections of the article (early/middle/late). This yields topics that can change vocabulary by section while keeping a single topic space for the whole document set.

- **Inputs:** `prep$documents`, `prep$vocab`, `prep$meta`, and `K`.
- **What it does:** Fits STM with `prevalence = ~ scale(position) + scale(length)` and `content = ~ section` (factor with levels early/middle/late). Uses spectral initialization for stability.
- **Outputs:** `stm_fit` (the fitted model) and `K_stm` (topics actually learned; guards downstream code if some topics collapse).

```
# Fit STM on the same counts/vocab used by LDA
# - documents/vocab/meta: identical preprocessed inputs from prepDocuments(...)
# - K: requested number of topics (may be adjusted by the model if some die out)
# - prevalence: lets topic proportions vary with position in article and paragraph length
#   (scale() standardizes these predictors for numerical stability)
# - content: allows each topic's word usage to differ across the 'section' factor
#   (early/middle/late thirds of the article)
# - init.type="Spectral": robust initialization recommended by the stm authors
stm_fit <- stm(
  documents  = prep$documents,
  vocab      = prep$vocab,
  K          = K,
  prevalence = ~ scale(position) + scale(length),
  content    = ~ section,
  data       = prep$meta,
  init.type  = "Spectral"
)
```

```
## Beginning Spectral Initialization
##   Calculating the gram matrix...
##   Finding anchor words...
##      .....
##   Recovering initialization...
##      ........
## Initialization complete.
## ..................................................................................................
## Completed E-Step (0 seconds).
## ..................................................................................................
## Completed M-Step.
## Completing Iteration 1 (approx. per word bound = -6.625)
## ..................................................................................................
## Completed E-Step (0 seconds).
## ..................................................................................................
## Completed M-Step.
## Completing Iteration 2 (approx. per word bound = -6.262, relative change = 5.483e-02)
## ..................................................................................................
## Completed E-Step (0 seconds).
## ..................................................................................................
```

```
## Completed M-Step.
## Completing Iteration 3 (approx. per word bound = -6.211, relative change = 8.078e-03)
## ..........................................................................................
## Completed E-Step (0 seconds).
## ..........................................................................................
## Completed M-Step.
## Completing Iteration 4 (approx. per word bound = -6.208, relative change = 4.455e-04)
## ..........................................................................................
## Completed E-Step (0 seconds).
## ..........................................................................................
## Completed M-Step.
## Completing Iteration 5 (approx. per word bound = -6.208, relative change = 7.740e-05)
## Topic 1: three, pretty, decades, wanted, part
##  Topic 2: hard, kurzweil, let's, believes, thinking
##  Topic 3: improve, matter, internet, everything, anything
##  Topic 4: bad, two, create, bostrom, way
##  Topic 5: help, process, yet, read, advanced
## Aspect 1: currently, collective, brain's, intense, physics
##  Aspect 2: confident, near, materials, material, standing
##  Aspect 3: code, malicious, empathy, none, skills
## ..........................................................................................
## Completed E-Step (0 seconds).
## ..........................................................................................
## Completed M-Step.
## Completing Iteration 6 (approx. per word bound = -6.204, relative change = 5.898e-04)
## ..........................................................................................
## Completed E-Step (0 seconds).
## ..........................................................................................
## Completed M-Step.
## Completing Iteration 7 (approx. per word bound = -6.203, relative change = 1.875e-04)
## ..........................................................................................
## Completed E-Step (0 seconds).
## ..........................................................................................
## Completed M-Step.
## Completing Iteration 8 (approx. per word bound = -6.202, relative change = 1.479e-04)
## ..........................................................................................
## Completed E-Step (0 seconds).
## ..........................................................................................
## Completed M-Step.
## Completing Iteration 9 (approx. per word bound = -6.201, relative change = 2.474e-04)
## ..........................................................................................
## Completed E-Step (0 seconds).
## ..........................................................................................
## Completed M-Step.
## Completing Iteration 10 (approx. per word bound = -6.200, relative change = 1.719e-04)
## Topic 1: three, wanted, part, probably, humanity
##  Topic 2: believes, kurzweil, hard, let's, thinking
##  Topic 3: question, power, matter, anything, everything
##  Topic 4: using, bostrom, bad, two, way
##  Topic 5: help, process, earth, whole, see
## Aspect 1: instantly, completely, caliber, living, works
##  Aspect 2: underestimating, optimistic, discuss, confident, individual
##  Aspect 3: takeoff, empathy, fermi, pulling, code
## ..........................................................................................
```

```
## Completed E-Step (0 seconds).
## ..........................................................................................................
## Completed M-Step.
## Model Converged
```

```
# Record the number of topics actually present in the fit
# (useful if some requested topics collapse or are pruned)
K_stm <- ncol(stm_fit$theta)
```

This is how STM uses covariates:

What "prevalence" means here. The `prevalence` formula controls the topic proportions per document (how much of each topic shows up in a paragraph). In the metadata, `position` is the exact paragraph index in the article (1, 2, 3, ...), and `length` is the token count of that paragraph. Writing `~ scale(position) + scale(length)` tells STM to use a logistic-normal regression where those two predictors explain why some paragraphs lean more toward certain topics than others. `scale()` simply centers and standardizes the predictors to help estimation (it doesn't change their meaning).

What "content" means here. The `content` formula controls the wording of topics, not how much of a topic a document has. `section` is a categorical factor we created by slicing `position` into terciles: "early", "middle", "late." With `content = ~ section`, `section` is a content covariate. STM learns different word distributions for the same topic depending on the section. For example, a "risk" topic might emphasize general setup terms early, debate words in the middle, and consequence terms late.

Why both `position` and `section`? They play different roles. `position` is a numeric, fine-grained signal that lets topic proportions drift smoothly across the article; `section` is a coarse, categorical signal that lets the **vocabulary** of each topic shift between early/middle/late. *content* (section) = changes **which** words a topic uses by location; *prevalence* (position/length) = changes in **how much** of each topic appears in a paragraph.

9) STM Labels (content-aware SAGE vs. FREX)

This chunk builds compact topic labels for the STM fit. If the model includes a content covariate (`section`), it uses `sageLabels()` and extracts *marginal* word lists (content-averaged), preferring **FREX** and falling back to **prob/lift/score** when needed. If there is no content covariate, it uses `labelTopics()`, which exposes **frex/prob/lift/score** at the top level. The returned matrices are normalized to "one row per topic," then the top words are concatenated into a single string per topic. The caption records which path was used.

- **Inputs:** `stm_fit`, `N_TOP_WORDS` (and `K_stm`); derives a `has_content` flag from `stm_fit$beta$logbeta`.
- **What it does:** Detects whether the STM was fit **with a content covariate** and chooses the right label extractor.
  - **With content:** calls `sageLabels(stm_fit, n=...)` and pulls top words from `labs$marginal` (prefers **FREX**, falls back to **prob/lift/score**).
  - **Without content:** calls `labelTopics(stm_fit, n=...)` and pulls **frex** (falls back to **prob/lift/score**).
  - Normalizes shapes (transposes if needed), concatenates each topic's top words into a single string, and builds `top_terms_stm = tibble(topic, top_words)`.
  - Renders a kable with a caption that reflects the path used: "SAGE marginal FREX" when content is present, "FREX" otherwise.
- **Outputs:** `top_terms_stm` tibble and a formatted **topic-labels table** that works for both STM setups.
- **Used next by:** Quick topic inspection/naming; no downstream dependencies.

```
# Detect whether the STM was fit WITH a content covariate.
# Multiple beta components imply multiple content levels.
has_content <- {
```

```r
  lb <- stm_fit$beta$logbeta
  is.list(lb) && length(lb) > 1
}

# Get label object:
# - With content: SAGE labels (content-aware); returns $marginal and $cov.betas
# - Without:      labelTopics() (classic FREX/prob/lift/score at top level)
labs <- if (has_content) {
  sageLabels(stm_fit, n = N_TOP_WORDS)
} else {
  labelTopics(stm_fit, n = N_TOP_WORDS)
}

# Turn 'labs' into a K-row character vector of "w1 w2 ... wN" per topic.
# Handles both SAGE (labs$marginal$frex/prob/lift/score) and labelTopics
# (labs$frex/prob/lift/score). Transposes if topics are in columns.
get_words <- function(labs, K) {
  # SAGE path: prefer marginal FREX; otherwise fall back to prob/lift/score
  if (!is.null(labs$marginal)) {
    cand <- list(labs$marginal$frex, labs$marginal$prob,
                 labs$marginal$lift, labs$marginal$score)
    for (x in cand) if (!is.null(x)) {
      m <- as.matrix(x)
      if (nrow(m) != K && ncol(m) == K) m <- t(m)   # ensure rows = topics
      return(apply(m, 1, function(row) paste(row, collapse = " ")))
    }
    stop("SAGE labels found, but no marginal word matrices were present.")
  }

  # No-content path: labelTopics fields live at the top level
  cand <- list(labs$frex, labs$prob, labs$lift, labs$score)
  for (x in cand) if (!is.null(x)) {
    m <- as.matrix(x)
    if (nrow(m) != K && ncol(m) == K) m <- t(m)     # ensure rows = topics
    return(apply(m, 1, function(row) paste(row, collapse = " ")))
  }

  stop("No word lists found in labs object.")
}

# Build the table used in later displays (topic index + concatenated top words)
top_terms_stm <- tibble::tibble(
  topic     = seq_len(K_stm),
  top_words = get_words(labs, K_stm)
)

# Caption documents which extractor was used
caption <- if (has_content) {
  "STM: Top 10 words per topic (SAGE marginal FREX)"
} else {
  "STM: Top 10 words per topic (FREX)"
}
```

```r
# Render a compact labels table
top_terms_stm |>
  kbl(caption = caption, booktabs = TRUE, longtable = TRUE, escape = TRUE,
      align = c("c","l")) |>
  kable_styling(latex_options = c("striped","scale_down")) |>
  column_spec(1, width = "1.2cm") |>
  column_spec(2, width = "12cm")
```

Table 4: STM: Top 10 words per topic (SAGE marginal FREX)

topic	top_words
1	brain probably far 're humanity new three point fact first
2	computer hard machine right might world kurzweil get median likely
3	question take back times everything actually great even time power
4	asi bostrom way species 'll may think level future artificial
5	ani earth smarter goal will nanobots every process quickly see

Results above STM's marginal FREX lists sharpen the same broad story. Topic 1 foregrounds brain/humanity and general setup terms. Topic 2 emphasizes computers/machines with Kurzweil and assessment words like median/likely. Topic 3 is "questions/time/power." Topic 4 is the ASI/Bostrom/species/future line. Topic 5 pulls ANI/goal/nanobots/process.

SAGE SAGE stands for Sparse Additive Generative (models of text), from Eisenstein, Ahmed, and Xing (2011). SAGE models each topic's word usage as a background word distribution plus a sparse set of additive deviations. The words with the largest positive deviations are the most characteristic of that topic. In **stm**, `sageLabels()` uses this idea to produce topic word lists: it highlights words that are not just frequent, but distinctive relative to the background, and (when we have a content covariate) it can report both marginal lists and content-specific lists. Compared with raw probability lists, SAGE-style labels usually read more like true "signatures" of a topic.

What are "marginal" word lists? In an STM with a content covariate (e.g., `section` = early/middle/late), the model learns a separate word distribution for each topic within each content level. A **marginal word list** is the single, overall word list for a topic obtained by **marginalizing over the content variable**—i.e., averaging the content-specific word weights across levels (typically weighted by how common those levels are in the corpus). It's called *marginal* because it comes from the marginal (content-averaged) topic–word distribution, not the distributions conditioned on a specific content level. Intuitively: estimated with the covariate, summarized as if we didn't condition on it.

Why "prefer FREX" and fall back to prob/lift/score? Different rankings highlight different kinds of words:

- **FREX** balances **frequency** (words that actually appear often in the topic) and **exclusivity** (words that are distinctive to the topic). This usually yields the most **label-friendly** set—specific but not obscure, so it's preferred.
- **prob** ranks by raw topic probability. It's clear but can surface generic terms that also occur in other topics.
- **lift** favors words that are rare overall but common in the topic, so it highlights distinctive terms; sometimes these can be a bit niche.
- **score** is another distinctiveness-leaning ranking that often sits between prob and lift.

The code tries marginal FREX first. If that field isn't available in the object returned by `sageLabels()/labelTopics()`, it falls back to marginal **prob**, then **lift**, then **score**, so we always get a sensible list even if some fields are missing.

10) STM: Top 2 Paragraphs per Topic

This mirrors the LDA "top documents" step but uses STM's **document–topic weights** to rank paragraphs. Topics are first ordered by prevalence (mean weight across documents), then the top N paragraphs for each topic are pulled with `top_docs_for_topic()`. `orig_ids` maps rows in the STM fit back to the original paragraph indices so snippets display the correct text. The result is a compact table for quick inspection and side-by-side comparison with LDA.

- **Inputs:** `stm_fit$theta`, `orig_ids`, `paragraphs`.
- **What it does:** Same idea as LDA top docs, but using STM's topic–document probabilities:
  - Orders topics by **prevalence** under STM.
  - Shows the **top 2 paragraphs** for each STM topic (with snippets).
- **Outputs:** A **"Top paragraphs per STM topic"** table.
- **Used next by:** Interpretation.

```r
# Document-topic weights from the STM fit
theta_stm   <- stm_fit$theta

# Order topics by prevalence (mean document-topic weight), most prevalent first
k_order_stm <- order(colMeans(theta_stm), decreasing = TRUE)

# For each topic (in prevalence order), collect the top-N paragraphs and add a rank-based topic index
top_docs_stm <- purrr::map_dfr(seq_along(k_order_stm), function(i) {
  k_orig <- k_order_stm[i]  # original topic index in the STM fit
  top_docs_for_topic(
    theta_mat = theta_stm,
    k         = k_orig,
    topn      = N_TOP_DOCS,
    map_ids   = orig_ids,      # map back to original paragraph IDs for correct snippets
    paras     = paragraphs
  ) |>
    dplyr::mutate(topic = i, .before = 1)  # label topics by prevalence rank: 1..K
})

# Render a consistent, readable table (topic, Doc ID, prob, snippet)
fmt_table(
  top_docs_stm,
  caption    = sprintf("STM: Top %d paragraphs per topic (prevalence order)", N_TOP_DOCS),
  align      = c("c","c","c","l"),
  widths     = WIDTHS_4,
  latex_opts = c("striped","hold_position","scale_down")
)
```

Table 5: STM: Top 2 paragraphs per topic (prevalence order)

topic	Doc ID	prob	snippet
1	276	0.567	When you try to achieve a long-reaching goal, you often aim for several subgoals along the way that will help you get to the final goal—the stepping stones to your goal. The official name for such a stepping stone is an instrumental goal. And again, if you don…

1	41	0.545	Cars are full of ANI systems, from the computer that figures out when the anti-lock brakes should kick in to the computer that tunes the parameters of the fuel injection systems. Google's self-driving car, which is being tested now, will contain robust ANI sys...
2	25	0.492	3) Our own experience makes us stubborn old men about the future. We base our ideas about the world on our personal experience, and that experience has ingrained the rate of growth of the recent past in our heads as "the way things happen." We're also limited ...
2	141	0.488	Müller and Bostrom also asked the experts how likely they think it is that we'll reach ASI A) within two years of reaching AGI (i.e. an almost-immediate intelligence explosion), and B) within 30 years. The results:4
3	14	0.386	Kurzweil suggests that the progress of the entire 20th century would have been achieved in only 20 years at the rate of advancement in the year 2000—in other words, by 2000, the rate of progress was five times faster than the average rate of progress during th...
3	136	0.376	So the median participant thinks it's more likely than not that we'll have AGI 25 years from now. The 90% median answer of 2075 means that if you're a teenager right now, the median respondent, along with over half of the group of AI experts, is almost certain...
4	240	0.417	What excites the Robotica team so much is that Turry is getting noticeably better as she goes. Her initial handwriting was terrible, and after a couple weeks, it's beginning to look believable. What excites them even more is that she is getting better at getti...
4	8	0.397	And then what if, after dying, he got jealous and wanted to do the same thing. If he went back 12,000 years to 24,000 BC and got a guy and brought him to 12,000 BC, he'd show the guy everything and the guy would be like, "Okay what's your point who cares." For...
5	70	0.401	Here's something we know. Building a computer as powerful as the brain is possible—our own brain's evolution is proof. And if the brain is just too complex for us to emulate, we could try to emulate evolution instead. The fact is, even if we can emulate a brai...
5	302	0.357	Goals like those won't suffice. So what if we made its goal, "Uphold this particular code of morality in the world," and taught it a set of moral principles. Even letting go of the fact that the world's humans would never be able to agree on a single set of mo...

The STM-ranked anchor paragraphs mirror the LDA themes but in STM's prevalence order. Topic 1 links "stepping-stone/instrumental" goals with concrete ANI systems in cars. Topic 2 contrasts human bias about growth rates with the Müller-Bostrom timing survey. Topic 3 brings back Kurzweil's acceleration argument alongside AGI median-year estimates. Topic 4 splits between Turry's improving handwriting and a time-travel thought experiment. Topic 5 ties brain-level feasibility arguments to goal-specification/morality.

11) STM: Wording by Content Level (Perspectives)

This section contrasts per-level wordings of each STM topic. Because the model was fit with a content covariate (section = early/middle/late), STM stores a separate topic-by-vocabulary matrix for each level. Sorting each matrix's rows (topics) by their largest log word-weights yields top words per topic for each level, showing how the same topic shifts its wording between early, middle, and late sections.

- **Inputs:** `stm_fit` (needs a content covariate with levels, e.g., early/middle/late).
- **What it does:** Shows **contrasting word lists** across content levels for each topic (e.g., words that are more characteristic in "early" vs "late" sections).
- **Why:** Helps see how content covariates change topic wording.
- **Outputs:** A **perspectives/contrasts** table (topic × contrast with exemplar words).
- **Used next by:** Interpretation.

```
# Vocabulary and factor levels for the content covariate
vocab     <- stm_fit$vocab
lvl_names <- levels(prep$meta$section)

# For content models, stm_fit$beta$logbeta is a list:
# one K×V matrix per content level (rows = topics, cols = vocab terms).
logbetas  <- stm_fit$beta$logbeta

# Given a K×V log-weight matrix for a level, return the top-n words per topic,
# concatenated as a single string per topic (one string per row).
get_top <- function(logbeta, n = N_TOP_WORDS) {
  apply(logbeta, 1, function(row) {
    # order indices of vocab by decreasing weight, take top-n, map to tokens, join with spaces
    paste(vocab[order(row, decreasing = TRUE)][seq_len(min(n, length(row)))],
          collapse = " ")
  })
}

# Compute a list of character vectors: one vector per level, length = K topics (native STM order)
by_level <- lapply(logbetas, get_top, n = N_TOP_WORDS)

# Assemble a table with one column per content level (early/middle/late), native topic order
persp <- tibble::tibble(topic = 1:K_stm)
for (i in seq_along(lvl_names)) {
  persp[[ lvl_names[i] ]] <- by_level[[i]]
}

# Reorder topics by prevalence so this table matches other STM/LDA tables
# (prevalence = mean document-topic weight across documents)
k_order_stm <- order(colMeans(stm_fit$theta), decreasing = TRUE)
persp <- persp |>
  dplyr::slice(k_order_stm) |>
  dplyr::mutate(topic = dplyr::row_number(), .before = 1)  # renumber 1..K in prevalence order

# Render a compact contrasts table: each row = a topic; columns = level-specific top words
persp |>
  kbl(caption = "STM: Top 10 words per topic by content level (prevalence order)",
      booktabs = TRUE, longtable = TRUE, escape = TRUE,
      align = c("c", rep("l", length(lvl_names)))) |>
  kable_styling(latex_options = c("striped","scale_down"), font_size = 9) |>
  column_spec(1, width = "1.2cm") |>
```

```
  column_spec(2:(length(lvl_names) + 1), width = "3.2cm")
```

Table 6: STM: Top 10 words per topic by content level (prevalence order)

topic	early	middle	late
1	intelligence will ani agi able smarter systems build see understand	will see nanobots balance agi extinction believe process something build	goal will intelligence turry's earth every final something superintelligent intelligent
2	way future think years asi 'll rate artificial computers century	asi 'll think species future bostrom two side way nanotech	asi way 'll bostrom smart think robotica ever may bad
3	computer might world machine get progress happen hard right kurzweil	kurzweil world believes hard get median tripwire right likely happen	turry get system computer becomes life right thinking world likely
4	time evolution even times power take back cps actually everything	time body question even beam everything anything don't power matter	even great time friendly better take don't post become black
5	brain 're far general first new part try point three	're first part far new three put brain probably fact	turry 're first new programmed far probably spider humanity handwriting

By section, wording shifts as expected. Early columns emphasize setup like "intelligence/able/build" and "time/evolution." Middle columns lean into evaluation—Bostrom/ASI and survey markers such as "median," along with "right/hard." Late columns move to mechanisms and consequences, surfacing "nanobots," "robotica," and "handwriting," plus goal/program language.

12) Four Labeling Criteria (No Content Fit)

This section refits STM **without** a content covariate to get a single, clean set of per-topic word rankings. It then contrasts the four common label criteria—**Prob**, **FREX**, **Lift**, **Score**—side by side. Using a no-content fit avoids per-level wording effects and makes the differences between ranking schemes easier to see. Topics are ordered by **prevalence** (mean document–topic weight) for consistent numbering.

```
# Refit STM WITHOUT a content covariate so we get a single set of lists
# (Prob / FREX / Lift / Score) per topic, unconfounded by per-level wording.
invisible(capture.output({
  stm_nocontent <- stm(
    documents  = prep$documents,
    vocab      = prep$vocab,
    K          = K,
    prevalence = ~ scale(position) + scale(length),  # allow topic proportions to vary by position/leng
    data       = prep$meta,
    init.type  = "Spectral",
    seed       = 99
  )
}, type = "output"))  # silence sampler output in the knit

# Order topics by prevalence (mean document-topic weight) for consistency with other tables
K_nc       <- ncol(stm_nocontent$theta)
k_order_nc <- order(colMeans(stm_nocontent$theta), decreasing = TRUE)
```

```r
# Pull the four ranking lists (each highlights a different notion of "top words")
labs4 <- labelTopics(stm_nocontent, n = N_TOP_WORDS)
# - labs4$prob : highest probability words within each topic
# - labs4$frex : words balancing frequency and exclusivity (label-friendly)
# - labs4$lift : words unusually concentrated in the topic vs. overall corpus
# - labs4$score: another distinctiveness-leaning ranking

# Helper: convert a labelTopics field into per-topic strings, respecting prevalence order.
# Handles both matrix (topics in rows OR columns) and list forms defensively.
col_strings <- function(x, k_order) {
  if (is.null(x)) return(rep("", length(k_order)))
  if (is.list(x)) {
    # List of length K: each element is a character vector of top words for a topic
    return(vapply(x[k_order], function(v) paste(v, collapse = " "), ""))
  }
  m <- as.matrix(x)
  if (ncol(m) == length(k_order)) {
    # Topics are columns: reorder columns by k_order, paste down each column
    apply(m[, k_order, drop = FALSE], 2, function(col) paste(col, collapse = " "))
  } else if (nrow(m) == length(k_order)) {
    # Topics are rows: reorder rows by k_order, paste across each row
    apply(m[k_order, , drop = FALSE], 1, function(row) paste(row, collapse = " "))
  } else {
    # Fallback: dimensions unexpected-return in given orientation
    apply(m, 2, function(col) paste(col, collapse = " "))
  }
}

# Build a compact table (prevalence-ordered topic index + the four label criteria)
terms_4 <- tibble::tibble(
  topic = seq_along(k_order_nc),                          # 1..K in prevalence order
  prob  = col_strings(labs4$prob,  k_order_nc),
  frex  = col_strings(labs4$frex,  k_order_nc),
  lift  = col_strings(labs4$lift,  k_order_nc),
  score = col_strings(labs4$score, k_order_nc)
)

# Render a side-by-side comparison of the four criteria (prevalence order)
terms_4 |>
  kbl(caption = "STM (no content): Top 10 words - Prob / FREX / Lift / Score (prevalence order)",
      booktabs = TRUE, longtable = TRUE, escape = TRUE,
      align = c("c","l","l","l","l")) |>
  kable_styling(latex_options = c("striped","scale_down"), font_size = 9) |>
  column_spec(1, width = "1.2cm") |>
  column_spec(2:5, width = "3.2cm")
```

Table 7: STM (no content): Top 10 words — Prob / FREX / Lift / Score (prevalence order)

topic	prob	frex	lift	score

1	agi asi years will species ’ll bostrom happen likely progress	species likely progress year century general rate growth three median	asked caliber fast growth likelihood major malicious median movie progress	progress rate century median history years year tripwire love agi
2	’re asi think right kurzweil might life future good actually	body means thought camp big topic aging anxious avenue ’re	big camp civilizations coming confident corner death fermi found gonna	camp future kurzweil body thinkers dying going aging kurzweil’s inevitable
3	time will first something machine superintelligence power beam intelligence question	beam friendly guy concept black place superintelligence step machine bring	friendly guy intense main advent ago beam beyond black blue	beam guy god concept time cps black friendly chimp impact
4	turry goal intelligence ani system new systems also way will	goal ani system systems better process turry’s working nanobots turn	allow chess goal killing market nanobots plan process skills starts	goal turry ani turry’s note systems robotica final explosion programmed
5	computer brain asi way able get far even smart build	brain wouldn’t nanotech trying kinds spider problem run anyway brain’s	anyway brain’s building couldn’t perfect alien architecture became billion billions	brain spider emulate able nanotech brain’s tarantula customers giving build

This no-content STM run shows how the four ranking schemes surface different facets of the same topics.

- **Prob** lists the highest-mass, on-theme words first. It’s great for the gist but often a bit generic (e.g., “intelligence,” “machine,” “time,” “year”).
- **FREX** balances frequency with exclusivity, so it tends to elevate crisp, label-friendly cues that aren’t drowned across topics (e.g., survey/timing terms or proper names when present).
- **Lift** amplifies tokens that are disproportionately concentrated in a topic relative to the whole corpus; that’s where rare but signature words pop up (think niche jargon, scenario terms, or named entities).
- **Score** lands between FREX and Prob in feel: more distinctive than raw probability but less quirky than Lift, often keeping summary-sounding verbs/nouns that read naturally in prose. Read across each topic row: the overlap between Prob and FREX gives the core label, while Lift/Score add the tell-tale “badges” that help you distinguish near-neighbors.

In short, use **FREX** for the label you print, sanity-check it against **Prob** for coverage, and mine **Lift/Score** for one or two distinctive add-ons when topics feel too similar.

- If a word is **high in Prob and FREX**, it’s part of the core theme—good label material.
- If it appears **only in Prob**, it’s likely generic glue (keep for interpretation, rarely for the printed label).
- If it appears **only in Lift**, it’s a sharp discriminator—great as a parenthetical qualifier (“AGI timelines (survey/median)”, “Goal-specification (tripwire)”), but avoid using Lift-only terms as the whole label.
- If **Score** agrees with FREX, that reinforces distinctiveness; if Score leans back toward Prob, the topic’s vocabulary is more shared, so rely on FREX + top paragraphs to avoid vague labels.
- When two topics look alike under Prob, check **Lift** for rare signals (proper names, scenario words) and **FREX** for balanced exclusives; those usually separate them immediately.